

1. What is Apache Kafka?

A: Apache Kafka is an open-source **distributed streaming platform** designed to handle high volumes of data feeds in real-time. It acts as a massive, durable, and highly available **commit log** that allows applications to publish, subscribe to, store, and process streams of records.

2. What are some common use cases of Kafka?

A: Common use cases include **real-time data pipelines** (moving data between systems), **activity tracking** (logging user clicks on a website), **metric collection**, and **event sourcing**. It is widely used as the central nervous system for large-scale, event-driven microservices architectures.

3. How does Kafka differ from traditional messaging systems?

A: Traditional messaging systems (like RabbitMQ) delete messages after they are consumed and often prioritize queues over permanent storage. Kafka differs by treating messages as a **durable, persistent log**; messages are stored for a configured retention period and are not deleted immediately after consumption. It is designed for high throughput and horizontal scalability.

4. What components make up the Kafka architecture?

A: The core components are the **Broker** (the servers that store the data), the **Producer** (the applications that send data), the **Consumer** (the applications that read data), and **ZooKeeper** (or the newer Kafka Quorum Controller, used for cluster management and coordination).

5. What is a Kafka Topic?

A: A **Topic** is a category or feed name to which records are published. Topics are the way Kafka logically organizes data. Producers write data to topics, and consumers read data from topics.

6. How do you create a topic in Kafka?

A: You create a topic using the **kafka-topics.sh** command-line utility, specifying the topic name, the number of partitions, and the replication factor. In modern Spring Boot applications, the topic can often be created automatically on startup via configuration.

7. How can topics be partitioned and why is this important?

A: Topics are split into **partitions**, which are the fundamental units of concurrency and parallelism in Kafka. Partitioning is important because it allows the data log to be distributed across multiple brokers, enabling **horizontal scaling** and handling massive data loads.

8. What happens when a topic is replicated in Kafka?

A: When a topic is replicated, the data in each partition is copied to a configured number of other brokers. This redundancy ensures that if one broker fails, the data is not lost, and a **replica** can be promoted to become the new leader, maintaining **high availability**.

9. Explain the role of Zookeeper in Kafka.

A: **ZooKeeper** is historically used for managing the Kafka cluster. Its role is to handle **broker registration**, maintain metadata (like configuration, topics, and partitions), and perform **leader election** when a broker or partition leader fails. It serves as the single source of truth for the cluster state.

10. Why is Zookeeper critical for Kafka?

A: ZooKeeper is critical because it ensures **cluster coordination and consistency**. Without it, brokers would not know which topics exist, which partitions they are responsible for, or which broker is the current partition leader. (Note: In Kafka 3.x and later, ZooKeeper is being replaced by the built-in Quorum Controller for simpler architecture.)

11. What would happen if Zookeeper were to fail?

A: If ZooKeeper were to fail, the existing Kafka cluster would continue to run and serve data. However, the cluster would become **read-only and unstable**. Brokers would lose the ability to elect new leaders, register new brokers, create new topics, or handle configuration changes until ZooKeeper is restored.

12. How does Kafka handle Zookeeper outages?

A: When a ZooKeeper outage occurs, Kafka maintains its existing operations and leadership elections temporarily halt. Brokers rely on their last known state. Once ZooKeeper is restored,

Kafka brokers re-establish connections and reconcile any changes, ensuring the cluster state is updated and stabilized.

13. What are Kafka Producers and Consumers?

A: Producers are client applications that write data (records) to Kafka topics. **Consumers** are client applications that read data from Kafka topics. They operate independently of each other.

14. How do producers send data to Kafka?

A: Producers serialize the data and send records to a Kafka broker. The producer determines which partition the message should go to, often based on the **message key**. The record is then written sequentially to the end of that partition's log.

15. What are some of the strategies consumers use to read data from Kafka?

A: Consumers pull messages from Kafka partitions based on their **offset** (the unique sequence ID of the message). They read records in the order they were written. Consumers commit their current offset to Kafka or ZooKeeper/Controller periodically to track their progress.

16. How can consumer groups enhance the scalability of Kafka?

A: Consumer groups enhance scalability by allowing multiple consumers to share the workload of reading messages from a topic. Each partition in the topic is assigned to only one consumer within the group. If you add more consumers than partitions, the extra consumers remain idle but act as backups.

17. What is the role of replication in Kafka?

A: The role of **replication** is to provide **fault tolerance** and **high availability**. By copying each partition's data to several brokers, replication ensures that if the leader broker fails, a replica can quickly take over leadership, allowing consumers and producers to continue operations without data loss or interruption.

18. What is the significance of the "acknowledgement" setting in producers?

A: The "acknowledgement" (**acks**) setting controls the durability guarantee when a producer sends a message. Setting **acks=0** means the producer doesn't wait for confirmation (fastest but risky). Setting **acks=all** means the producer waits until the leader has committed the message and all in-sync replicas have confirmed receipt (slowest but safest).

19. How does Kafka ensure data is not lost? (basic answer: ACKS, replication)

A: Kafka ensures data is not lost through two primary mechanisms: setting producer **acknowledgements (acks) to all** (requiring full confirmation) and using **replication** to create redundant copies of the data across multiple brokers. This combination guarantees high durability.

20. What is Kafka Connect and why is it useful?

A: **Kafka Connect** is a tool used for streaming data between Kafka and other data systems. It is useful because it allows developers to configure **connectors** (pre-built components) to pull data from external sources (like a database or file system) or push data to external sinks (like Elasticsearch) without writing custom code.

21. How do you scale Kafka Connect?

A: You scale Kafka Connect by running multiple **workers**. The workload (the connectors) is distributed automatically across the available worker processes. Scaling is managed by increasing the number of worker nodes, which handle fault tolerance and distributing connectors among themselves.

22. How do you ensure messages with the same key are processed in order?

A: You ensure order for messages with the same key by guaranteeing they are all sent to the **same partition**. Kafka guarantees that all messages within a single partition will be processed sequentially and in order.

23. What steps would you take if Kafka consumers are lagging behind? (basic explanation)

A: I would take three steps: **1)** Increase the number of consumers in the consumer group up to the number of partitions (to fully utilize parallelism). **2)** Increase the number of partitions (if the lag persists and is acceptable to the application logic). **3)** Optimize the consumer processing logic to run faster.

24. What strategies would you use if Kafka messages exceed the default size limit?

A: First, I would increase the **broker's configuration limits** (message.max.bytes). If the message size is still too large, I would avoid storing the entire payload in Kafka. Instead, I would store the large data in an external store (like S3) and send only a **reference (URL/key)** to that data through the Kafka message.

25. How do you ensure data consistency when multiple consumers read from the same topic?

A: Data consistency is ensured by organizing consumers into a **consumer group**. Within a group, each partition is assigned to only one consumer, which prevents duplicate processing. If the application needs to process the same data for different purposes, those consumers should be in **separate** consumer groups.

26. What happens if a Kafka partition leader fails, and how does Kafka handle leader election?

A: If a partition leader fails, the partition becomes temporarily unavailable. Kafka (via the Controller) immediately triggers a **leader election** among the remaining **in-sync replicas (ISRs)**. The broker that is successfully elected becomes the new leader, and service is restored.